Using this function, we send it four pieces of information: the BlinkM's address followed by the values for hue, saturation, and brightness. The function will send the character "h" to the BlinkM followed by the HSB values, and the BlinkM will work out the math for fading from one color to another.

Well, that was a fairly lengthy overview of how the code works, but we should back up now and look at what's involved in writing and working with functions like those in our project example.

Writing Functions

So far in this chapter, we have seen some additional functions that are part of the main Arduino library, as well as some that we have written. We've also looked at how these functions can extend the capabilities of our code. Functions provide the programmer with the ability to compartmentalize chunks of code that are related by a very specific purpose. We are not limited to only those functions that are provided in the Arduino programming environment, as in fact, writing functions is the bread and butter of writing source code for C and something that any seasoned Arduino programmer will do to make their code function better, take less memory space, make the code better organized, reduce the possibilities for errors, and generally make it easier to read.

The uses for functions are really endless and we have already been using them from the very beginning. Maybe we need to perform certain arithmetic conversions on a particular analog reading. We could write a specific function to perform these tasks for us and return the finished converted values back to the main `loop()` function. Maybe we want to turn on a set of digital outputs all at the same time or in a sequence any time a certain condition has been met. We could make a function that we could pass a condition or value to and have the function decide what to do with it. For example, let's say we wanted to create a function that will turn on and off the pin 13 LED. That could look something like the following:

```
void blinkLED() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

Now, every time we call the `blinkLED()` function in our sketch, the pin 13 LED will turn on and off once before returning to whatever the code was doing before. Now, let's looks at exactly what's involved in making these functions work.

Declaring Functions

To begin working with a new function, we need to first declare it. A **function declaration** involves establishing the function's **return data type**, the function's name, and any **parameters** that are being passed to the function bracketed by parenthesis. We will discuss returns and parameters in a moment, but for now you should be comfortable with what a function declaration looks like. In our last example, the function declaration is fairly simple, as follows:

```
void blinkLED() {
```

The keyword `void` is our function's return data type. The name given to our function in this case is `blinkLED`. This provides an understandable name for our function that tells us at a quick glance what we can expect the function to do. Since we are not talking about parameters at the moment, the